

List Comprehensions

Introduction

List Comprehension is a concise and Pythonic way to create lists.

Instead of using a `for` loop and the `append()` method, we can create a new list in a single line of code.

It makes the code shorter, cleaner, and easier to read.

Advantages

- Short and readable code
- Faster than using loops with `append()`
- Creates a new list in a single line
- Supports conditions (`if` and `if-else`)
- Can be used with nested loops

Normal Approach (Using for Loop)

Syntax

```
Python
result = []

for item in iterable:
    result.append(expression)
```

Example

```
Python
squares = []

for i in range(1, 6):
    squares.append(i * i)

print(squares)
```

```
# Output:  
# [1, 4, 9, 16, 25]
```

Using List Comprehension

Syntax

```
Python  
[expression for item in iterable]
```

Example

```
Python  
squares = [i * i for i in range(1, 6)]  
  
print(squares)  
  
# Output:  
# [1, 4, 9, 16, 25]
```

List Comprehension Syntax

1. Basic Syntax

Syntax

```
Python  
[expression for item in iterable]
```

Components

- **expression** → Value that will be added to the new list.
- **item** → Current element from the iterable.
- **iterable** → A sequence such as a list, tuple, string, set, or range.

Example

```
Python
numbers = [1, 2, 3, 4, 5]

result = [x * x for x in numbers]

print(result)

# Output:
# [1, 4, 9, 16, 25]
```

2. List Comprehension with if Condition

Syntax

```
Python
[expression for item in iterable if condition]
```

Example

```
Python
result = [x for x in range(1, 11) if x % 2 == 0]

print(result)

# Output:
# [2, 4, 6, 8, 10]
```

3. List Comprehension with if-else

Syntax

```
Python
[true_value if condition else false_value for item in iterable]
```

Example

Python

```
result = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 6)]
```

```
print(result)
```

Output:

```
# ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

4. Nested List Comprehension

Syntax

Python

```
[expression for item1 in iterable1 for item2 in iterable2]
```

Example

Python

```
result = [(x, y) for x in [1, 2] for y in [3, 4]]
```

```
print(result)
```

Output:

```
# [(1, 3), (1, 4), (2, 3), (2, 4)]
```

5. Nested List Comprehension with Condition

Syntax

Python

```
[expression for item1 in iterable1 for item2 in iterable2 if condition]
```

Example

Python

```
result = [(x, y) for x in range(3) for y in range(3) if x != y]
```

```
print(result)
```

```
# Output:
```

```
# [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

List Comprehension Examples

Example 1: Squares

Python

```
numbers = [1, 2, 3, 4, 5]
```

```
result = [x * x for x in numbers]
```

```
print(result)
```

```
# Output:
```

```
# [1, 4, 9, 16, 25]
```

Example 2: Even Numbers

Python

```
result = [x for x in range(1, 11) if x % 2 == 0]
```

```
print(result)
```

```
# Output:
```

```
# [2, 4, 6, 8, 10]
```

Example 3: Uppercase Names

Python

```
names = ["teja", "ravi", "sneha"]
```

```
result = [name.upper() for name in names]
```

```
print(result)
```

```
# Output:
```

```
# ['TEJA', 'RAVI', 'SNEHA']
```

Example 4: String Lengths

Python

```
words = ["python", "java", "sql"]
```

```
result = [len(word) for word in words]
```

```
print(result)
```

```
# Output:
```

```
# [6, 4, 3]
```

Example 5: Add 10 to Every Number

Python

```
numbers = [5, 10, 15, 20]
```

```
result = [x + 10 for x in numbers]
```

```
print(result)
```

```
# Output:
```

```
# [15, 20, 25, 30]
```

Example 6: Convert Strings to Lowercase

Python

```
names = ["CODEGNAN", "PYTHON", "JAVA"]
```

```
result = [name.lower() for name in names]
```

```
print(result)
```

```
# Output:  
# ['codegna', 'python', 'java']
```

Example 7: First Letter of Each Word

```
Python  
words = ["Python", "Java", "SQL"]  
  
result = [word[0] for word in words]  
  
print(result)  
  
# Output:  
# ['P', 'J', 'S']
```

Example 8: Numbers Greater Than 50

```
Python  
numbers = [20, 55, 80, 45, 100]  
  
result = [x for x in numbers if x > 50]  
  
print(result)  
  
# Output:  
# [55, 80, 100]
```

Example 9: Even or Odd

```
Python  
numbers = [1, 2, 3, 4, 5]  
  
result = ["Even" if x % 2 == 0 else "Odd" for x in numbers]  
  
print(result)
```

```
# Output:  
# ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Example 10: Create a List of Tuples

```
Python  
numbers = [1, 2, 3, 4]  
  
result = [(x, x * x) for x in numbers]  
  
print(result)  
  
# Output:  
# [(1, 1), (2, 4), (3, 9), (4, 16)]
```

Nested Comprehensions

Introduction

A **Nested Comprehension** contains one loop inside another loop.

It is used when we need to iterate over multiple iterables, create combinations, flatten nested lists, or work with matrices.

Nested comprehensions are commonly used in data processing, matrix operations, and generating combinations.

Advantages

- Reduces multiple nested loops into a single line
- Produces cleaner and more readable code
- Useful for matrices and multidimensional data
- Efficient for generating combinations and Cartesian products

Nested Comprehension Syntax

1. Basic Nested Comprehension

Syntax

```
Python  
[expression for item1 in iterable1 for item2 in iterable2]
```

Example

```
Python  
result = [(x, y) for x in [1, 2] for y in [3, 4]]  
  
print(result)  
  
# Output:  
# [(1, 3), (1, 4), (2, 3), (2, 4)]
```

2. Nested Comprehension with Condition

Syntax

```
Python  
[expression for item1 in iterable1 for item2 in iterable2 if  
condition]
```

Example

```
Python  
result = [(x, y) for x in range(3) for y in range(3) if x != y]  
  
print(result)  
  
# Output:  
# [(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

3. Nested Comprehension with if-else

Syntax

```
Python
[true_value if condition else false_value
 for item1 in iterable1
 for item2 in iterable2]
```

Example

```
Python
result = ["Even" if (x + y) % 2 == 0 else "Odd"
          for x in range(2)
          for y in range(2)]

print(result)

# Output:
# ['Even', 'Odd', 'Odd', 'Even']
```

Nested Comprehension Examples

Example 1: Multiplication Table

```
Python
result = [i * j for i in range(1, 4) for j in range(1, 4)]

print(result)

# Output:
# [1, 2, 3, 2, 4, 6, 3, 6, 9]
```

Example 2: Cartesian Product

```
Python
colors = ["Red", "Blue"]
sizes = ["S", "M", "L"]
```

```
result = [(c, s) for c in colors for s in sizes]
```

```
print(result)
```

```
# Output:
```

```
# [('Red', 'S'), ('Red', 'M'), ('Red', 'L'),  
#  ('Blue', 'S'), ('Blue', 'M'), ('Blue', 'L')]
```

Example 3: Matrix Flattening

Python

```
matrix = [  
    [1, 2],  
    [3, 4],  
    [5, 6]  
]
```

```
result = [item for row in matrix for item in row]
```

```
print(result)
```

```
# Output:
```

```
# [1, 2, 3, 4, 5, 6]
```

Example 4: All Possible Pairs

Python

```
numbers = [1, 2, 3]
```

```
result = [(x, y) for x in numbers for y in numbers]
```

```
print(result)
```

```
# Output:
```

```
# [(1,1), (1,2), (1,3),  
#  (2,1), (2,2), (2,3),  
#  (3,1), (3,2), (3,3)]
```

Example 5: Pairs with Different Values

Python

```
result = [(x, y) for x in range(1, 4) for y in range(1, 4) if x  
!= y]
```

```
print(result)
```

Output:

```
# [(1,2), (1,3),
```

```
#  (2,1), (2,3),
```

```
#  (3,1), (3,2)]
```

Generators

Definition

A **Generator** is a special function that produces values **one at a time** instead of returning all values at once.

Generators save memory because values are generated only when they are needed.

They are especially useful when working with large datasets.

Advantages

- Memory efficient
- Faster for large datasets
- Values are generated only when needed
- Suitable for processing large files and streams
- Improves performance

Normal Function

Syntax

Python

```
def function_name():  
    return value
```

Example

Python

```
def numbers():  
    return [1, 2, 3, 4, 5]  
  
print(numbers())  
  
# Output:  
# [1, 2, 3, 4, 5]
```

Explanation:

The entire list is created and stored in memory before it is returned.

Generator Function

Syntax

Python

```
def function_name():  
    yield value1  
    yield value2  
    yield value3
```

Example

Python

```
def numbers():  
    yield 1  
    yield 2  
    yield 3  
    yield 4  
    yield 5  
  
print(numbers())  
  
# Output:  
# <generator object numbers at ...>
```

Explanation:

Instead of returning all values together, the generator produces one value at a time.

Generator vs Normal Function

Normal Function

Python

```
def numbers():  
    return [1, 2, 3, 4, 5]
```

- Returns all values together.
- Stores the complete list in memory.

Generator Function

Python

```
def numbers():  
    yield 1  
    yield 2  
    yield 3  
    yield 4  
    yield 5
```

- Produces one value at a time.
- Uses less memory.
- Remembers its previous state after every **yield**.

Yield Keyword

Definition

The **yield** keyword converts a normal function into a **generator function**.

Instead of returning all values at once like **return**, **yield** returns one value at a time and pauses the execution of the function.

When the next value is requested, execution resumes from where it was paused.

Syntax

Python

```
yield value
```

or

Python

```
yield expression
```

Example 1

Python

```
def display():  
    yield "Python"  
    yield "Java"  
    yield "SQL"
```

```
data = display()
```

```
print(next(data))  
print(next(data))  
print(next(data))
```

```
# Output:  
# Python  
# Java  
# SQL
```

Example 2

Python

```
def even_numbers():
```

```
for i in range(2, 11, 2):  
    yield i  
  
for num in even_numbers():  
    print(num)  
  
# Output:  
# 2  
# 4  
# 6  
# 8  
# 10
```

Example 3

```
Python  
def countdown():  
    for i in range(5, 0, -1):  
        yield i  
  
for num in countdown():  
    print(num)  
  
# Output:  
# 5  
# 4  
# 3  
# 2  
# 1
```

next() Function

Definition

The `next()` function retrieves the next value from a generator.

Each call to `next()` resumes execution from the previous `yield` statement.

Syntax

Python

```
next(generator)
```

Example

Python

```
def demo():  
    yield 10  
    yield 20  
    yield 30  
  
g = demo()  
  
print(next(g))  
print(next(g))  
print(next(g))  
  
# Output:  
# 10  
# 20  
# 30
```

Working of next()

Python

```
def demo():  
    yield 10  
    yield 20  
    yield 30
```

```
g = demo()

print(next(g))    # First yield
print(next(g))    # Second yield
print(next(g))    # Third yield
```

Execution Flow

1. Function starts execution.
2. `yield 10` returns **10** and pauses.
3. `next()` resumes execution.
4. `yield 20` returns **20** and pauses.
5. `next()` resumes execution.
6. `yield 30` returns **30** and pauses.
7. Generator finishes execution.

StopIteration Example

When all values in a generator have been consumed, calling `next()` again raises a `StopIteration` exception.

Example

```
Python
def demo():
    yield 10
    yield 20

g = demo()

print(next(g))
print(next(g))
print(next(g))
```

Output

```
None
10
```

20

StopIteration

Explanation

The generator contains only two values. After they are consumed, another call to `next()` raises `StopIteration`.

Generator Expressions

Introduction

A **Generator Expression** is a concise way to create generators.

It is similar to a **List Comprehension**, but uses **parentheses ()** instead of **square brackets []**.

Unlike list comprehensions, generator expressions do **not** create all values at once. They generate values only when required.

Generator Expression Syntax

Basic Syntax

```
Python
(expression for item in iterable)
```

Example

```
Python
numbers = (x * x for x in range(5))

print(numbers)

# Output:
# <generator object>
```

Generator Expression with if

Syntax

Python

```
(expression for item in iterable if condition)
```

Example

Python

```
numbers = (x for x in range(10) if x % 2 == 0)
```

```
for num in numbers:  
    print(num)
```

Output:

0

2

4

6

8

Generator Expression with if-else

Syntax

Python

```
(true_value if condition else false_value for item in iterable)
```

Example

Python

```
numbers = ("Even" if x % 2 == 0 else "Odd" for x in range(1, 6))
```

```
for num in numbers:  
    print(num)
```

Output:

Odd

Even

```
# Odd
# Even
# Odd
```

Nested Generator Expression

Syntax

```
Python
(expression for item1 in iterable1 for item2 in iterable2)
```

Example

```
Python
pairs = ((x, y) for x in [1, 2] for y in [3, 4])

for pair in pairs:
    print(pair)

# Output:
# (1, 3)
# (1, 4)
# (2, 3)
# (2, 4)
```

Difference Between Functions and Generators

Feature	Function	Generator
Keyword Used	<code>return</code>	<code>yield</code>
Execution	Executes completely	Executes partially
Memory Usage	Higher	Lower

Stores All Values	Yes	No
Produces Values	All at once	One at a time
Return Type	Actual Value	Generator Object
State	Does not remember previous execution	Remembers previous execution after each <code>yield</code>
Suitable For	Small datasets	Large datasets